Real-Time Obstacle Detection and Warning System

Using FreeRTOS on Arduino Mega 2560

Submitted by:
Lakhdari Anis Charef Eddine

Embedded Systems Project
January 2, 2026

1 Project Overview

This project is an **Obstacle Detection and Warning System** built using a multi-tasking architecture. It utilizes an ultrasonic sensor to monitor distance and a servo motor for scanning. The system is managed by the **FreeRTOS** kernel, ensuring that safety alarms and motor control run as independent, prioritized tasks.

2 Bill of Materials (BOM) & Libraries

2.1 Hardware Components

- **Microcontroller:** Arduino Mega 2560
- **Ultrasonic Sensor:** HC-SR04
- **Actuator:** Standard Servo Motor (SG90)
- **Visual Output:** LCD 16x2 (HD44780)
- **Audio Output:** Active Buzzer (5V)
- **Simulation Input:** Voltage Source Knob (0-5V) for distance simulation.

2.2 Software Libraries

- `Arduino_FreeRTOS.h` (Kernel management)
- `LiquidCrystal.h` (LCD interface)

3 How to Use & Simulation

1. **Power On:** The LCD initializes and displays "System 200cm...".
2. **Simulation Mode:** In SimulIDE, adjust the distance by varying the voltage at the **V-in** pin of the sensor using a **Voltage Source Knob**.
3. **Normal State:** If the simulated distance is > 200 **cm**, the servo sweeps 0–180°.
4. **Detection State:** If the simulated distance drops < 200 **cm**:
   - The servo stops instantly.
   - The buzzer pulses (50ms on / 1000ms off).
   - The LCD updates to "!! TOO CLOSE !!".
5. **Recovery:** Increase the simulated voltage (distance) to resume normal operation.

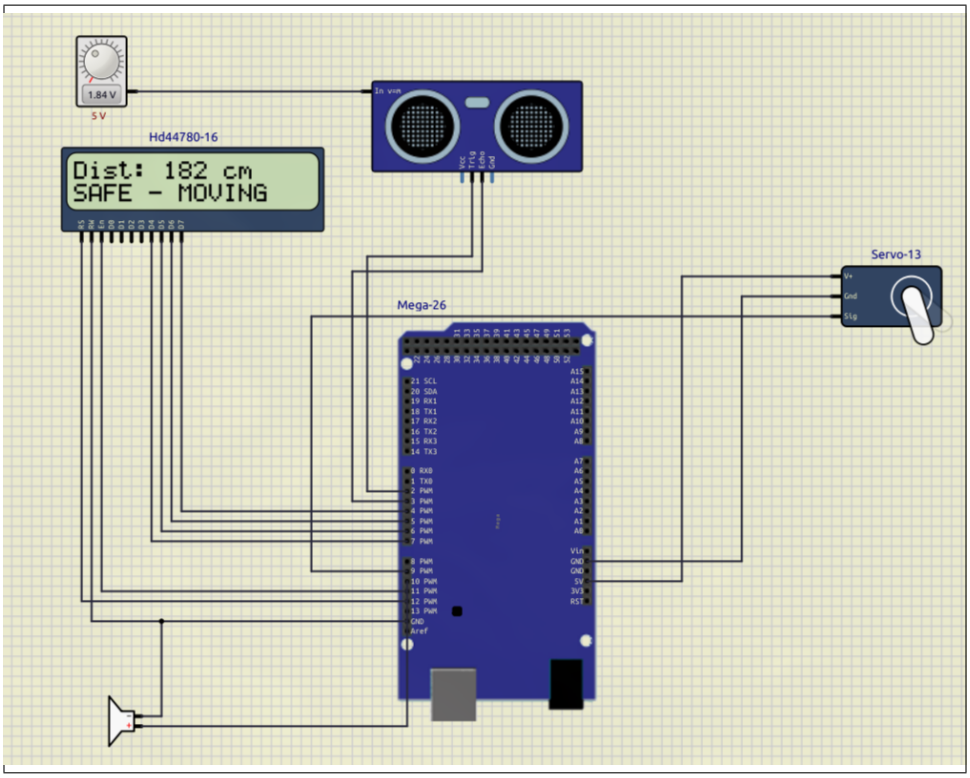

Figure 1: SimulIDE Simulation showing Voltage Source Knob for distance control.

4 Distance Calculation Logic

The system calculates distance based on the **Time of Flight** of sound waves.

1. **Triggering:** `TaskSensor` sends a 10μs high pulse to the `TRIG` pin.
2. **Echo:** The HC-SR04 sends ultrasonic bursts and sets the `ECHO` pin to HIGH until the sound returns.
3. **Measurement:** We use `pulseIn()` to measure the duration (t) in microseconds.
4. **Formula:** The speed of sound is ≈ 0.034 cm/μs. Dividing by 2 for the round trip:

$$\text{Distance (cm)} = \frac{t \times 0.034}{2} \tag{1}$$

5 FreeRTOS Task Priorities

Priority is assigned via `xTaskCreate`. A higher number indicates higher urgency.

Task	Priority	Function
TaskSound	3	**Highest:** Immediate alarm trigger.
TaskDisplay	2	**Medium:** LCD updates and status.
TaskSensor	1	**Low:** Background distance monitoring.
TaskMotor	1	**Low:** Physical servo movement.

Table 1: FreeRTOS Task Priority Table

6 Memory Management: Why Mega 2560?

- **The Problem:** The Arduino Uno (2KB SRAM) is insufficient for FreeRTOS. Creating four task stacks led to immediate **Stack Overflows**, causing the kernel to crash.
- **The Solution:** The **Arduino Mega 2560 (8KB SRAM)** provides 4x more memory. This allows the FreeRTOS heap to accommodate all task stacks comfortably.

7 Technical Challenges & Optimizations

- **Timer Conflict Resolution:** `Servo.h` and FreeRTOS both use **Timer 1**. I resolved this by removing the library and using **Manual Software PWM** via `digitalWrite` inside the motor task.
- **Stack Optimization:** Stacks were tuned to save RAM: Sensor (128 words), Display (192 words), and Sound (100 words).
- **F() Macro Usage:** Static strings are stored in **Flash memory** instead of SRAM to prevent memory exhaustion.